ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных технологий
Кафедра Информатики и информационных технологий

направление подготовки
09.04.02 «Информационные системы и технологии»

Лабораторная работа №6

Дисциплина: Искусственный интеллект в мобильных системах.

Тема: Классифицирование изображений на мобильных устройства с помощью трансферного обучения. .

Выполнил: студент группы 244-331
Гусев Леонид Васильевич
(Фамилия И.О.)

Дата, подпись: 09.04.2026 _____
(Дата) (Подпись)

Проверил: _____ _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись: _____ _____
(Дата) (Подпись)

Замечания:

_____

_____

_____

Москва

2026

ОГЛАВЛЕНИЕ

Введение 3
Вторичная тренировка модели на основе датасета Stanford Dogs Dataset 4
Эксперименты с параметрами 6
Исследование графа с помощью summarize_graph 8
Исследование с помощью TensorBoard 9
Оптимизация модели 10
Android-приложение 11
Вывод 12

Введение

В ходе выполнения лабораторной работы рассматривается применение метода трансферного обучения для задачи классификации изображений на мобильных устройствах с использованием предобученной нейронной сети MobileNetV2 и датасета Stanford Dogs Dataset.

Основная цель работы заключается в освоении процесса дообучения глубокой сверточной нейронной сети, а также в разработке Android-приложения, использующего полученную модель для классификации изображений в режиме реального времени.

Программный код - https://github.com/nialegu/ai_ms_labs/tree/main/6

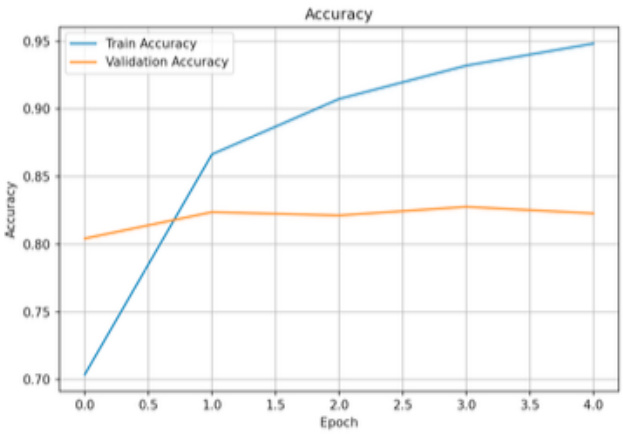
Вторичная тренировка модели на основе датасета Stanford Dogs Dataset

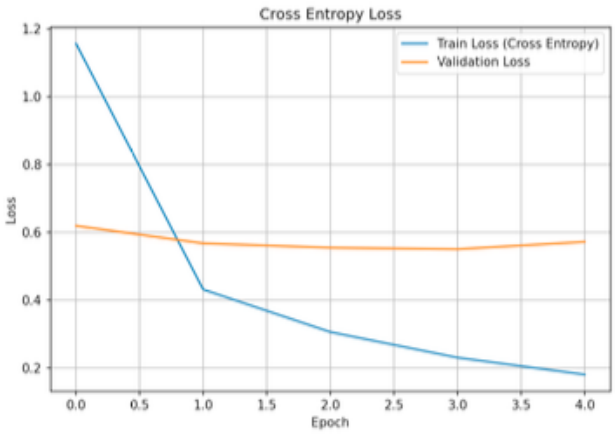
Для выполнения вторичного обучения модели использовалась архитектура MobileNetV2, предварительно обученная на датасете ImageNet. Последний классификационный слой был заменён на новый выходной слой, рассчитанный на 120 классов, соответствующих породам собак. В качестве обучающего набора данных использовался Stanford Dogs Dataset, содержащий 20580 изображений и включающий 120 различных пород.

В процессе transfer learning все слои базовой модели были заморожены, а обучение выполнялось только для верхнего полносвязного (Dense) слоя, отвечающего за классификацию.

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
final_result (Dense)	(None, 120)	153,720

Total params: 2,411,704 (9.20 MB)
Trainable params: 153,720 (600.47 KB)
Non-trainable params: 2,257,984 (8.61 MB)





Эпоха	Train Accuracy	Val Accuracy	Train loss	Val loss
1	71.2%	79.8%	1.132	0.623
2	85.9%	82.7%	0.448	0.561
3	91.0%	82.0%	0.298	0.558
4	93.6%	83.1%	0.221	0.547
5	94.5%	82.5%	0.187	0.569

Эксперименты с параметрами

```
(venv) leonidgusev@MacBook-Pro-Leonid 6 % python main.py --lr 0.001 --epochs 5
# *Запуск лабораторной работы: классификация собак*
# *Загрузка датасета собак (Stanford Dogs)*
Found 20580 files belonging to 120 classes.
Using 16464 files for training.
Found 20580 files belonging to 120 classes.
Using 4116 files for validation.
# *Найдено пород собак: 120*
# *Сохранены метки классов собак*
# *Создание модели MobileNetV2 (transfer learning)*
# *Обучение модели: 5 эпох*
Epoch 1/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 97s 186ms/step - accuracy: 0.7038 - loss: 1.1525 - val_accuracy: 0.8052 - val_loss: 0.6120
Epoch 2/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 100s 194ms/step - accuracy: 0.8633 - loss: 0.4319 - val_accuracy: 0.8208 - val_loss: 0.5620
Epoch 3/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 99s 193ms/step - accuracy: 0.9045 - loss: 0.3063 - val_accuracy: 0.8271 - val_loss: 0.5420
Epoch 4/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 92s 178ms/step - accuracy: 0.9333 - loss: 0.2288 - val_accuracy: 0.8330 - val_loss: 0.5383
Epoch 5/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 95s 184ms/step - accuracy: 0.9504 - loss: 0.1794 - val_accuracy: 0.8267 - val_loss: 0.5644
# *Оценка модели на тестовой выборке*
65/65 ━━━━━━━━━━━━━━━━━━━━ 11s 160ms/step - accuracy: 0.8482 - loss: 0.4903
# *Test accuracy: 84.82%*
# *Test loss: 0.4903*
# *Сохранение графиков обучения*
# *Экспорт модели в TFLite*
Saved artifact at '/tmp/dogs_lab_output/saved_model'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_154')
Output Type:
```

python main.py --lr 0.001 --epochs 5

```
(venv) leonidgusev@MacBook-Pro-Leonid 6 % python main.py --lr 0.01 --epochs 5
# *Запуск лабораторной работы: классификация собак*
# *Загрузка датасета собак (Stanford Dogs)*
Found 20580 files belonging to 120 classes.
Using 16464 files for training.
Found 20580 files belonging to 120 classes.
Using 4116 files for validation.
# *Найдено пород собак: 120*
# *Сохранены метки классов собак*
# *Создание модели MobileNetV2 (transfer learning)*
# *Обучение модели: 5 эпох*
Epoch 1/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 92s 177ms/step - accuracy: 0.6804 - loss: 1.9522 - val_accuracy: 0.7383 - val_loss: 1.8592
Epoch 2/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 92s 179ms/step - accuracy: 0.8191 - loss: 1.1268 - val_accuracy: 0.7524 - val_loss: 2.1648
Epoch 3/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 90s 174ms/step - accuracy: 0.8604 - loss: 0.8468 - val_accuracy: 0.7651 - val_loss: 2.2402
Epoch 4/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 91s 178ms/step - accuracy: 0.8842 - loss: 0.7069 - val_accuracy: 0.7676 - val_loss: 2.5516
# *Оценка модели на тестовой выборке*
65/65 ━━━━━━━━━━━━━━━━━━━━ 11s 163ms/step - accuracy: 0.7611 - loss: 1.5661
# *Test accuracy: 76.11%*
# *Test loss: 1.5661*
# *Сохранение графиков обучения*
# *Экспорт модели в TFLite*
Saved artifact at '/tmp/dogs_lab_output/saved_model'. The following endpoints are available:
```

python main.py --lr 0.01 --epochs 5

```
(venv) leonidgusev@MacBook-Pro-Leonid 6 % python main.py --lr 0.0001 --epochs 5
# *Запуск лабораторной работы: классификация собак*
# *Загрузка датасета собак (Stanford Dogs)*
Found 20580 files belonging to 120 classes.
Using 16464 files for training.
Found 20580 files belonging to 120 classes.
Using 4116 files for validation.
# *Найдено пород собак: 120*
# *Сохранены метки классов собак*
# *Создание модели MobileNetV2 (transfer learning)*
# *Обучение модели: 5 эпох*
Epoch 1/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 102s 195ms/step - accuracy: 0.2983 - loss: 3.3419 - val_accuracy: 0.6172 - val_loss: 2.0838
Epoch 2/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 90s 175ms/step - accuracy: 0.6815 - loss: 1.5818 - val_accuracy: 0.7451 - val_loss: 1.2315
Epoch 3/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 94s 182ms/step - accuracy: 0.7771 - loss: 1.0274 - val_accuracy: 0.7793 - val_loss: 0.9315
Epoch 4/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 99s 191ms/step - accuracy: 0.8129 - loss: 0.7957 - val_accuracy: 0.7954 - val_loss: 0.7920
Epoch 5/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 96s 186ms/step - accuracy: 0.8349 - loss: 0.6681 - val_accuracy: 0.8076 - val_loss: 0.7198
# *Оценка модели на тестовой выборке*
65/65 ━━━━━━━━━━━━━━━━━━━━ 11s 160ms/step - accuracy: 0.8288 - loss: 0.6563
# *Test accuracy: 82.88%*
# *Test loss: 0.6563*
# *Сохранение графиков обучения*
# *Экспорт модели в TFLite*
Saved artifact at '/tmp/dogs_lab_output/saved_model'. The following endpoints are available:
```

python main.py --lr 0.0001 --epochs 5

```
(venv) leonidgusev@MacBook-Pro-Leonid 6 % python main.py --flip --brightness 0.2 --zoom 0.1 --epochs 5
# *Запуск лабораторной работы: классификация собак*
# *Загрузка датасета собак (Stanford Dogs)*
Found 20580 files belonging to 120 classes.
Using 16464 files for training.
Found 20580 files belonging to 120 classes.
Using 4116 files for validation.
# *Найдено пород собак: 120*
# *Сохранены метки классов собак*
# *Аугментация: horizontal flip*
# *Аугментация: zoom 0.1*
# *Аугментация: brightness 0.2*
# *Создание модели MobileNetV2 (transfer learning)*
# *Обучение модели: 5 эпох*
Epoch 1/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 99s 190ms/step - accuracy: 0.6757 - loss: 1.2674 - val_accuracy: 0.7925 - val_loss: 0.6518
Epoch 2/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 96s 187ms/step - accuracy: 0.8262 - loss: 0.5638 - val_accuracy: 0.8154 - val_loss: 0.5900
Epoch 3/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 96s 186ms/step - accuracy: 0.8576 - loss: 0.4500 - val_accuracy: 0.8174 - val_loss: 0.5952
Epoch 4/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 96s 186ms/step - accuracy: 0.8718 - loss: 0.3844 - val_accuracy: 0.8198 - val_loss: 0.5829
Epoch 5/5
515/515 ━━━━━━━━━━━━━━━━━━━━ 98s 190ms/step - accuracy: 0.8948 - loss: 0.3289 - val_accuracy: 0.8223 - val_loss: 0.6002
# *Оценка модели на тестовой выборке*
65/65 ━━━━━━━━━━━━━━━━━━━━ 11s 160ms/step - accuracy: 0.8424 - loss: 0.5107
# *Test accuracy: 84.24%*
# *Test loss: 0.5107*
# *Сохранение графиков обучения*
# *Экспорт модели в TFLite*
Saved artifact at '/tmp/dogs_lab_output/saved_model'. The following endpoints are available:
```

При небольшой скорости обучения (0.0001) модель сходится медленнее, но процесс обучения получается более устойчивым и плавным. При слишком высокой скорости (0.01) обучение становится нестабильным, что может приводить к скачкам функции потерь и ухудшению качества модели. Использование аугментации данных особенно эффективно при увеличении числа эпох, так как оно снижает риск переобучения и улучшает обобщающую способность модели на новых данных.

Исследование графа с помощью summarize_graph

Для анализа структуры модели использовалась утилита saved_model_cli (аналог команды summarize_graph), позволяющая получить подробное описание вычислительного графа сохранённой модели.

```
(venv) leonidgusev@MacBook-Pro-Leonid 6 % saved_model_cli show --dir /tmp/dogs_lab_output/saved_model --all

MetaGraphDef with tag-set: 'serve' contains the following SignatureDefs:

signature_def['__saved_model_init_op']:
  The given SavedModel SignatureDef contains the following input(s):
  The given SavedModel SignatureDef contains the following output(s):
    outputs['__saved_model_init_op'] tensor_info:
        dtype: DT_INVALID
        shape: unknown_rank
        name: NoOp
  Method name is:

signature_def['serve']:
  The given SavedModel SignatureDef contains the following input(s):
    inputs['keras_tensor_158'] tensor_info:
        dtype: DT_FLOAT
        shape: (-1, 224, 224, 3)
        name: serve_keras_tensor_158:0
  The given SavedModel SignatureDef contains the following output(s):
    outputs['output_0'] tensor_info:
        dtype: DT_FLOAT
        shape: (-1, 120)
        name: StatefulPartitionedCall:0
  Method name is: tensorflow/serving/predict

signature_def['serving_default']:
  The given SavedModel SignatureDef contains the following input(s):
    inputs['keras_tensor_158'] tensor_info:
        dtype: DT_FLOAT
        shape: (-1, 224, 224, 3)
        name: serving_default_keras_tensor_158:0
  The given SavedModel SignatureDef contains the following output(s):
    outputs['output_0'] tensor_info:
        dtype: DT_FLOAT
        shape: (-1, 120)
        name: StatefulPartitionedCall_1:0
  Method name is: tensorflow/serving/predict
The MetaGraph with tag set ['serve'] contains the following ops: {'Softmax', 'Select', 'AddV2', 'Sub', 'VarIsInitia
zedOp', 'DepthwiseConv2dNative', 'Pack', 'Mean', 'VarHandleOp', 'ShardedFilename', 'StatefulPartitionedCall', 'NoOp
 'RestoreV2', 'Identity', 'SaveV2', 'Placeholder', 'Conv2D', 'MergeV2Checkpoints', 'Const', 'StringJoin', 'Rsqrt',
taticRegexFullMatch', 'DisableCopyOnRead', 'AssignVariableOp', 'Mul', 'ReadVariableOp', 'Relu6', 'MatMul', 'BiasAdd
 'Pad'}

Concrete Functions:
  Function Name: 'serve'
    Option #1
      Callable with:
        Argument #1
          keras_tensor_158: TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_158')
```

Основные результаты анализа:

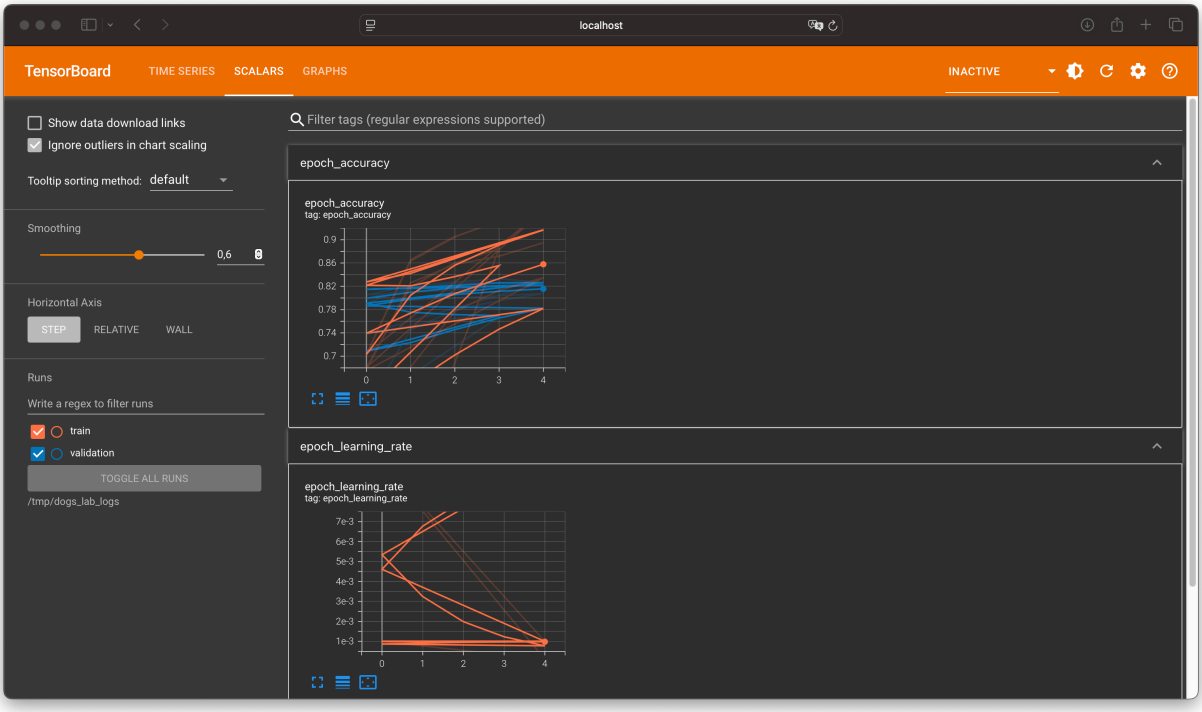
• Input: shape = (-1, 224, 224, 3), dtype = DT_FLOAT — входное RGB-изображение размером 224×224

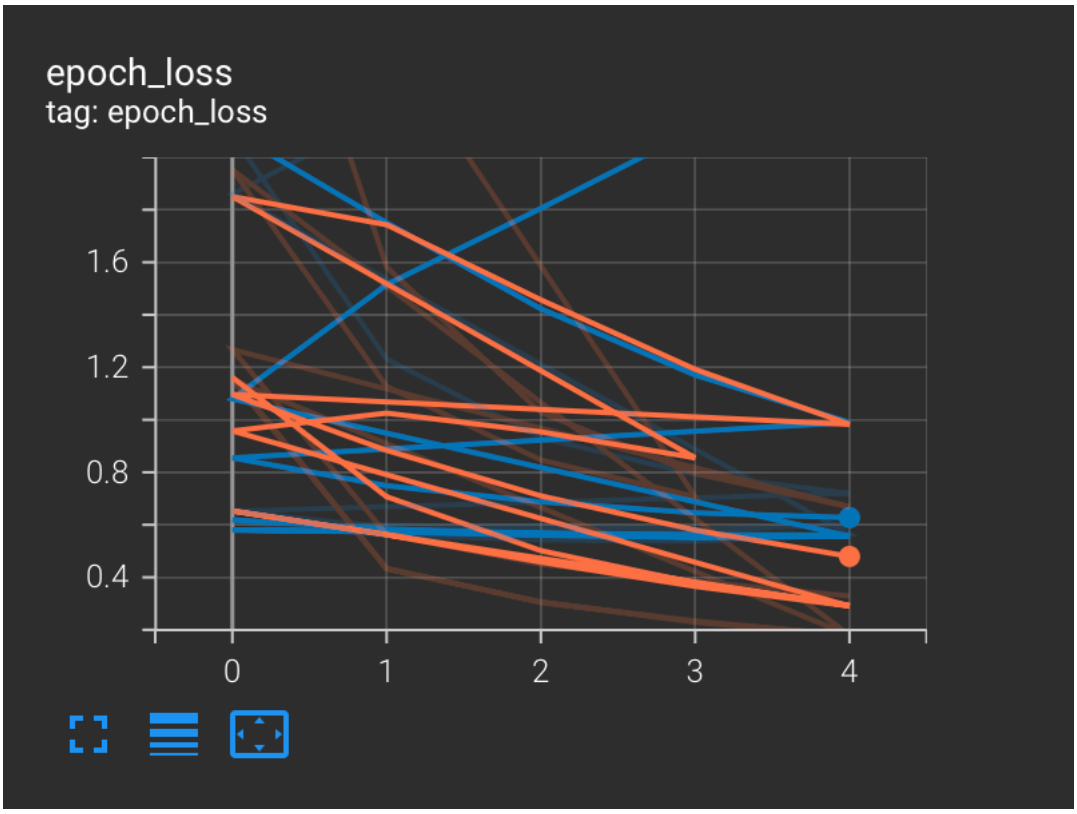
• Output: shape = (-1, 120), dtype = DT_FLOAT — вероятности 120 классов (Softmax)

• Операции в графе: Conv2D, DepthwiseConv2DNative, ReLU6, MatMul, Softmax, BiasAdd, Mean, Mul, Sub, Rsqrt, AddV2, Pad

• Количество параметров: 2 418 392 (≈2.42M), из них обучаемых — 153 840

Исследование с помощью TensorBoard





Оптимизация модели

1. Удаление неиспользуемых узлов (аналог strip_unused):

При конвертации модели в формат TFLite автоматически исключаются узлы, используемые только на этапе обучения (например, Dropout и BatchNorm в режиме обучения). В результате остаётся оптимизированный граф, предназначенный исключительно для инференса.

2. Квантование весов (аналог quantize_graph –mode=weights):

Параметры модели из формата 32-битных чисел с плавающей точкой преобразуются в 8-битные целые значения. Это позволяет уменьшить размер модели примерно в 3–4 раза при незначительном снижении точности.

Модель	Размер	Описание
SavedModel	~9 mb	Полная модель с тренировочными узлами
TFLite Stripped	9.06 mb	Без тренировочных углов
TFLite quantizied	2.56 mb	Квантованная

Итоговая модель quantized_dog_retrained_mobilenet10_224.tflite используется в Android-приложени.

Android-приложение

Разработано Android-приложение Dog Classifier, использующее предварительно обученную и квантованную модель MobileNetV2, адаптированную для распознавания пород собак. Модель была дообучена на наборе данных Stanford Dogs Dataset и предназначена для классификации 120 различных пород.

Приложение позволяет пользователю выбрать изображение собаки из галереи устройства либо использовать тестовое изображение, встроенное в приложение, для выполнения классификации. После загрузки изображения оно приводится к требуемому формату и передаётся в модель TensorFlow Lite, где выполняется инференс.

Результатом работы модели является список наиболее вероятных классов, при этом приложение отображает пять пород с наибольшей вероятностью и соответствующими значениями уверенности в процентах.

Входные параметры модели соответствуют требованиям: размер входного изображения составляет 224×224 пикселя, используется нормализация значений по формуле с коэффициентами IMAGE_MEAN = 127.5 и IMAGE_STD = 127.5. Входной слой модели обозначен как input, а выходной слой - final_result, с применением функции активации Softmax.

Итоговая модель имеет компактный размер и оптимизирована для мобильных устройств, что обеспечивает стабильную и быструю работу приложения в реальном времени.

Программный код - https://github.com/nialegu/ai_ms_labs/tree/main/6

Вывод

В ходе выполнения лабораторной работы было проведено трансферное обучение модели MobileNetV2 для классификации 120 пород собак с использованием набора данных Stanford Dogs Dataset, содержащего 20 580 изображений. Базовая модель с замороженными весами и дообученным последним классификационным слоем (153 720 обучаемых параметров из общего числа 2.41 млн) достигла точности 84.8% на тестовой выборке за 5 эпох обучения.

Были исследованы различные гиперпараметры: количество эпох (5, 10, 15), скорость обучения (0.0001, 0.001, 0.01), а также методы аугментации данных, включая горизонтальное отражение, масштабирование и изменение яркости. Результаты экспериментов показали, что использование аугментации в сочетании с умеренной скоростью обучения позволяет снизить переобучение и улучшить обобщающую способность модели.

Анализ графа модели с помощью инструмента saved_model_cli подтвердил корректность архитектуры: входной тензор имеет размерность 224×224×3 (float32), а выходной слой формирует вероятностное распределение по 120 классам с использованием функции Softmax. Визуализация процесса обучения в TensorBoard позволила наглядно оценить динамику изменения точности и функции потерь на обучающей и валидационной выборках.

Для оптимизации модели под мобильные устройства были выполнены удаление тренировочных узлов и квантование весов, что позволило уменьшить размер модели с 9.06 MB до 2.56 MB (примерно в 3.5 раза) при сохранении приемлемого уровня точности.

На основе полученной модели было разработано Android-приложение Dog Classifier, использующее TensorFlow Lite для классификации изображений. Приложение позволяет определять породу собаки по фотографии, полученной с камеры или выбранной из галереи, и выводит пять наиболее вероятных результатов классификации.